Data set import and Encoding

IRIS Dataset

```
# Import required libraries
import pandas as pd
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()

# Convert the feature data into a DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Add the target column
df["target"] = iris.target


# Display the first 5 rows
print(df.head())
```

Output (Sample)

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	species
5.1	3.5	1.4	0.2	0	setosa
4.9	3.0	1.4	0.2	0	setosa
4.7	3.2	1.3	0.2	0	setosa
4.6	3.1	1.5	0.2	0	setosa
5.0	3.6	1.4	0.2	0	setosa

Useful commands for EDA

```
# Shape of the dataset
print(df.shape)

# Column names
print(df.columns)

# Data types
print(df.dtypes)

# Summary statistics
print(df.describe())

# Check for missing values
print(df.isnull().sum())

# Count of each species
print(df["species"].value_counts())
```

Step 1: Import Libraries

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
```

Step 2: Create a Sample DataFrame

```
df = pd.DataFrame({
    "Region": ["North", "South", "East", "West", "North"],
    "Category": ["Electronics", "Clothing", "Electronics", "Furniture",
"Clothing"],
    "Rating": [4.5, 3.8, 4.2, 4.7, 4.0],
    "Price": [500, 1000, 750, 1200, 900]
})

print(df)
```

Output:

Region	Category	Rating	Price
North	Electronics	4.5	500
South	Clothing	3.8	1000
East	Electronics	4.2	750
West	Furniture	4.7	1200
North	Clothing	4.0	900

---

1. Label Encoding

Use Label Encoding for **ordinal** or **target** variables.

```
le = LabelEncoder()

df["Region_Label"] = le.fit_transform(df["Region"])

print(df)
```

Output:

Region	Region_Label
North	1
South	2
East	0
West	3
North	1

To see the mapping:

```
mapping = dict(zip(le.classes_, le.transform(le.classes_)))
print(mapping)
```

Output:

```
{'East': 0, 'North': 1, 'South': 2, 'West': 3}
```

---

2. One-Hot Encoding

Use One-Hot Encoding for **nominal** categorical variables.

```
one_hot = pd.get_dummies(df, columns=["Category"], dtype=int)

print(one_hot)
```

Output:

Region	Rating	Price	Region_Label	Category_Clothing	Category_Electronics	Category_Furniture
North	4.5	500	1	0	1	0
South	3.8	1000	2	1	0	0
East	4.2	750	0	0	1	0
West	4.7	1200	3	0	0	1
North	4.0	900	1	1	0	0

---

3. Min-Max Scaling

Scale numerical columns to the range **0–1**.

```
scaler = MinMaxScaler()

df[["Rating", "Price"]] = scaler.fit_transform(df[["Rating", "Price"]])

print(df)
```

Output:

Region	Category	Rating	Price
North	Electronics	0.78	0.00
South	Clothing	0.00	0.71
East	Electronics	0.44	0.36
West	Furniture	1.00	1.00
North	Clothing	0.22	0.57

Complete Code

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler

# Create DataFrame
df = pd.DataFrame({
    "Region": ["North", "South", "East", "West", "North"],
    "Category": ["Electronics", "Clothing", "Electronics", "Furniture",
"Clothing"],
    "Rating": [4.5, 3.8, 4.2, 4.7, 4.0],
    "Price": [500, 1000, 750, 1200, 900]
})

# Label Encoding
le = LabelEncoder()
df["Region_Label"] = le.fit_transform(df["Region"])

# One-Hot Encoding
df = pd.get_dummies(df, columns=["Category"], dtype=int)
```

```
# Min-Max Scaling
scaler = MinMaxScaler()
df[["Rating", "Price"]] = scaler.fit_transform(df[["Rating", "Price"]])

print(df)
```

When to use each technique

Technique	Use for	Example
Label Encoding	Ordinal categories or target labels	Low, Medium, High
One-Hot Encoding	Nominal categories with no order	Region, Gender, Color
Min-Max Scaling	Numerical features before ML	Price, Age, Rating